

## Mid Tearm

### Pdf - 01

#### 1. What is Microprocessors?

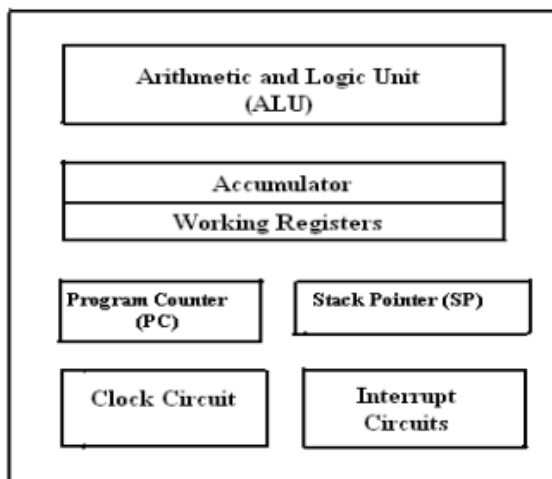
**Ans:** Microprocessor is the core of computer systems. A microprocessor is a multipurpose, programmable, clock-driven, register based electronic device that reads binary instructions from a storage device called memory.

#### 2. Why do we need to learn Microprocessors and Microcontrollers?

Today there is no electronic gadget on the earth which is designed without a Microcontroller. Ex: communication devices, digital entertainment, portable devices etc...

- **Personal information products:** Cell phone, pager, watch, pocket recorder, calculator.
- **Laptop components:** mouse, keyboard, modem, fax card, sound card, battery charger
- **Home appliances:** Door lock, alarm clock, air conditioner, Tv remote etc.
- **Industrial equipment:** Temperature/pressure controllers, Counters, timers, RPM Controllers
- **Toys:** video games, cars, dolls, etc.

#### 3. Explain Microprocessor with block diagram.



**i. Arithmetic and Logic Unit (ALU) :** It is the place where actual computations take place. Consist of circuits which performs arithmetic operations.

**ii. Control Unit (CU):** The coordination and control of a computer is the sole responsibility of the Control Unit.

**iii. Registers:** Registers are temporary storage locations inside microprocessor that holds data and address.

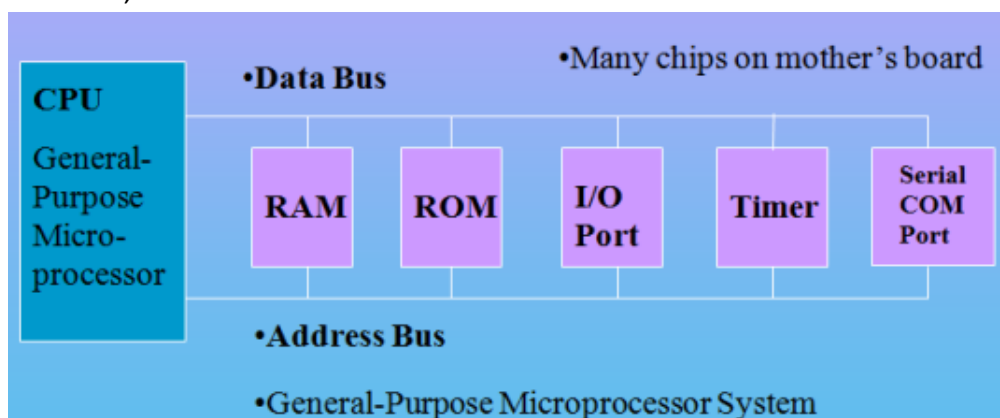
**iv. Program counter (PC):** It is commonly known as IP (Instruction Pointer) in 8086, is a register in microprocessor that contains the address of the next instruction to be executed.

**v. Stack Pointer (SP):** It is a small register that stores the address of the last program/instruction request in a stack.

**Fig. Block diagram of a Microprocessor.**

#### 4. Microprocessor:

- CPU for Computers
- No RAM, ROM, I/O on CPU chip itself
- Example: Intel's x86, Motorola's 680x0

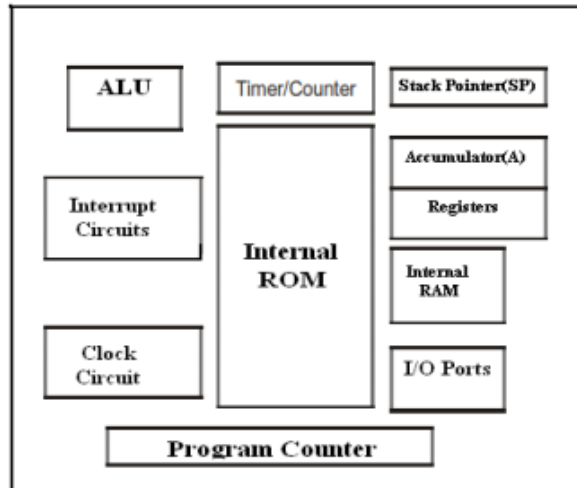


## 5. What is Microcontroller?

**Ans:** A microcontroller is a highly integrated single chip, which consists of on chip CPU (Central Processing Unit), RAM (Random Access Memory), EPROM/PROM/ROM (Erasable Programmable Read Only Memory), I/O (input/output) – serial and parallel ports, timers, interrupt controller.

**For example:** Intel 8051 is 8-bit microcontroller and Intel 8096 is 16-bit microcontroller.

## 6. Block Diagram of Microcontroller:



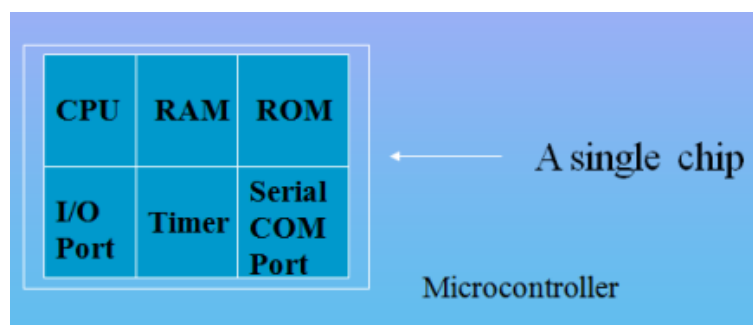
**Fig. Block Diagram of a Microcontroller**

## 7. Applications of Microcontrollers:

Cell phone, Pager, watch, calculator, video games, Alarm clock, air conditioner, Tv remote, Microwave oven, washing machines, robotic system, anti-lock braking system monitor.

## 8. Microcontroller:

- A smaller computer
- On-chip RAM, ROM, I/O ports...
- Example: Motorola's 6811, Intel's 8051, Zi log's Z8 and PIC



## 9. What is RISC (Reduced Instruction Set Computer)?

It stands for Reduced Instruction Set Computer. It is a type of microprocessor architecture that uses a small set of instructions of uniform length. These are simple instructions that are generally executed in one clock cycle. RISC chips are relatively simple to design and inexpensive. **Examples: SPARC, POWER PC etc.**

## 10. What is CISC (Complex Instruction Set Computer)?

It stands for Complex Instruction Set Computer. These processors offer the users, hundreds of instructions of variable sizes. CISC architecture includes a complete set of special-purpose circuits that carry out these instructions at a very high speed. CISC processors reduce the program size and hence lesser number of memory cycles are required to execute the programs. This increases the overall speed of execution.

**Examples: Intel architecture, AMD**

### 11. Distinguish with Microprocessor and Microcontroller.

| Microprocessor                                               | Microcontroller                                       |
|--------------------------------------------------------------|-------------------------------------------------------|
| CPU is stand-alone, RAM, ROM, I/O, timer is separate         | CPU, RAM, ROM, I/O and timer are all on a single chip |
| General-purpose                                              | Single-purpose (control-oriented)                     |
| Designer can decide on the amount of ROM, RAM and I/O ports. | Fixed amount of on-chip ROM, RAM, I/O ports           |
| High processing power                                        | Low processing power                                  |
| Typically, 16/32/64 – bit                                    | Typically, 8/16 bit                                   |
| Typically, deep pipeline (5-20 stages)                       | Typically, single-cycle/two-stage pipeline            |

### 12. Distinguish with RISC and CISC.

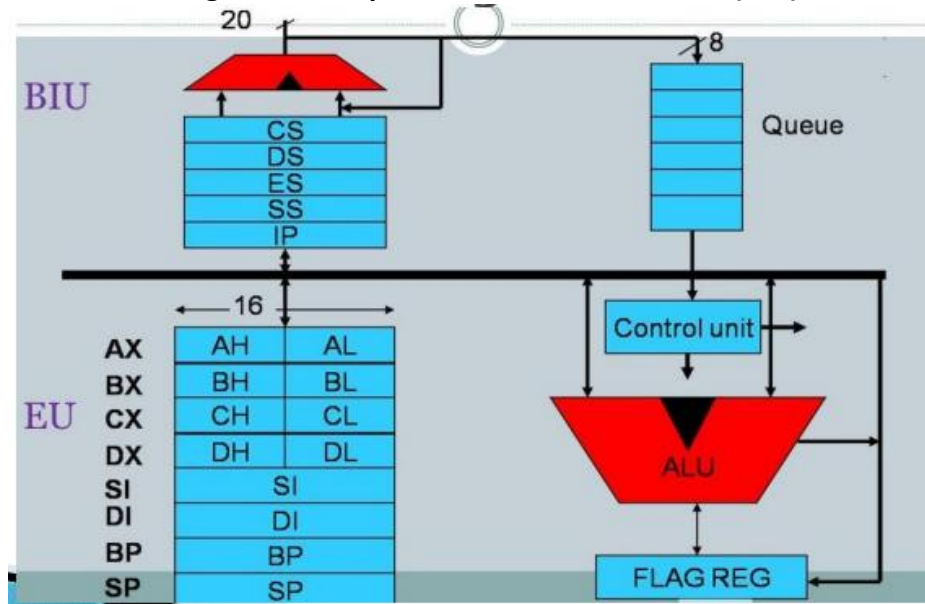
| RISC                                                                      | CISC                                                                      |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------|
| RISC stands for Reduced Instruction Set Computer. It's focus on software. | CISC stands for Complex Instruction Set Computer. It's focus on Hardware. |
| Uses only Hardwired control unit.                                         | Uses both hardwired and microprogrammed control unit.                     |
| Can perform only Register to Register Arithmetic operations               | Can perform REG to REG or REG to MEM or MEM to MEM                        |
| RISC is Reduced Instruction Cycle.                                        | CISC is Complex Instruction Cycle.                                        |
| Simple and limited addressing modes.                                      | Complex and more addressing modes.                                        |
| Code size is large                                                        | Code size is small                                                        |
| Here, addressing modes are less.                                          | Here, addressing modes are more.                                          |

## Pdf – 02

### 1. Explain the 8086 Microprocessor

- ✓ It is a 16-bit microprocessor
- ✓ It has 16-bit data bus
- ✓ It has 20-bit address bus
- ✓ 8086 CPU is divided into two independent functional parts.
- ✓ Bus interface unit (BIU)
- ✓ Execution unit (EU)

## 2. Draw the 8086 internal block diagram and explain the Bus Interface Unit(BIU) and Execution Unit(EU)



### \*\*\*Bus Interface Unit:

#### 1. Instruction Queue-

- ❖ 6-byte queue
- ❖ Prefetch instructions (FIFO)

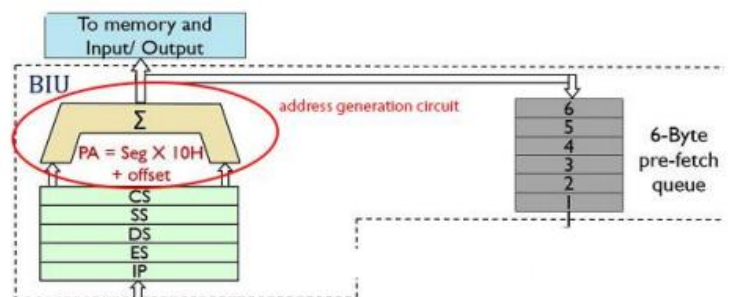
#### 2. Segment Register-

Start address of a data. It has 4 segment registers that holds address of instruction and data in memory.

- Code Segment register
- Data Segment register
- Extra Segment register
- Stack Segment register

#### 3. Instruction Pointer –

It's a 16 bit register that holds offset of the next instructions in the CS.



### \*\*\*Execution Unit in 8086:

This is an execution unit where instruction decoding and execution takes place.

**1. General Purpose Register :** Four 16 bit register divided into 2 parts low [8bit] and high [8bit] use for data handling.

- i. AX                      iii. CX
- ii. BX                    iv. DX

**2. Pointers register :** contain offset address.

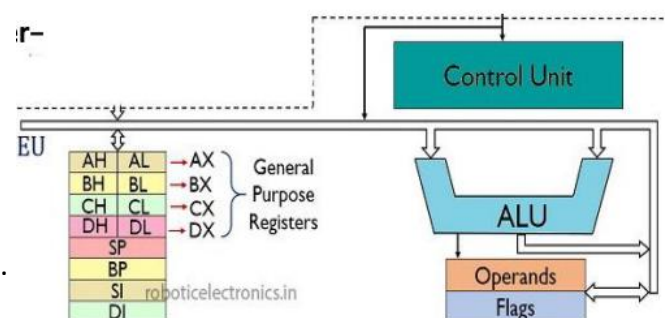
- SP(Stack pointer)- point to stack, top of stack segment.  
 $PA = [SS] * 10H + [SP]$
- BP(Base pointer)- use to access random location of stack.  
 $PA = [DS] * 10H + [BP]$

**3. Index register:**

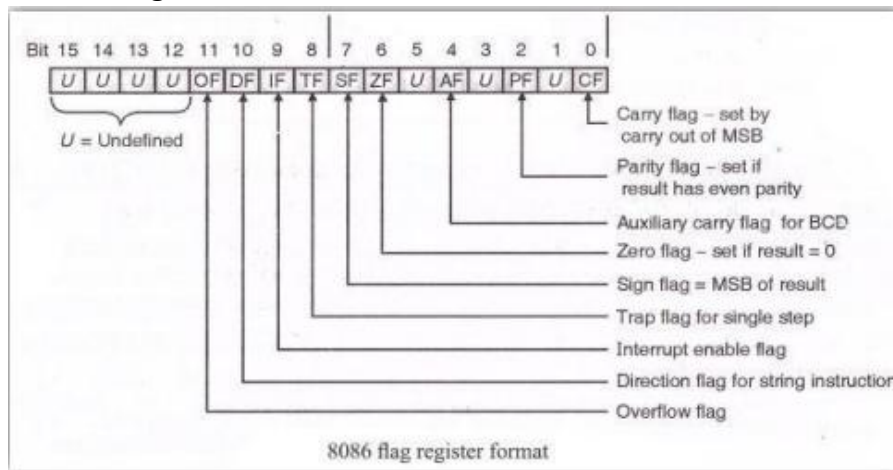
- DI (Destination Index)
- SI (Source Index)

**4. Temporary register:** Store temporary

**5. ALU :** Perform arithmetic and logical Operation.



### 3. Explain Flag registers with diagram/ format



The flag register is a 16-bit register in the Intel 8086 microprocessor that contains information about the state of the processor after executing an instruction. There are two types of flag register:

#### 1) conditional/status and 2) control flag

##### 1. Conditional/ Status flag:

- **Carry flag-** It is set whenever there is a carry or borrow out of the MSB (most significant bit) of a result. D7 bit for an 8-bit operation and D15 bit for a 16-bit operation.
- **Auxiliary carry flag-** this flag is set when ALU perform BCD operation of 8-bit and if result in a carry or borrow.
- **Parity flag-** It is set if the result has even parity. If parity is odd, PF is reset.
- **Zero flag-** This flag is set 1 when the result of ALU operation is zero else it is set to zero.
- **Sign flag-** It is set if the MSB of the result is 1.
- **Overflow flag-** It will be set if the result of a signed operation is too large to fit in the number of bits available to represent it.

##### 2. Control Flag:

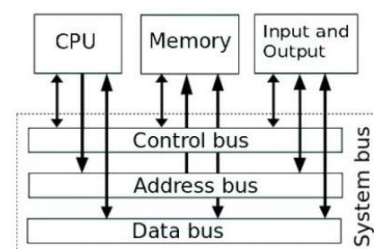
- **Trap flag-** It is used to set the trace mode i.e. start single stepping mode.
- **Interrupt flag-** this flag is used to enable or disable interrupt. It is set to 1 for interrupt enabled and set to 0 for interrupt disabled
- **Direction flag-** It is used in string operation.

### Pdf – 3

#### 1. Function of 8085 with diagram.

**Ans:**

- **Bit:** A bit is a single binary digit.
- **Word:** A word refers to the basic data size or bit size that can be processed by the arithmetic and logic unit of the processor. A 16-bit binary number is called a word in a 16-bit processor.
- **Bus:** A bus is a group of wires/lines that carry similar information.
- **System Bus:** The system bus is a group of wires/lines used for communication between the microprocessor.
- **Address Bus:** It carries the address, which is a unique binary pattern used to identify a memory location or an I/O port. For example, an eight-bit address bus has eight lines and thus it can address  $2^8 = 256$  different locations.
- **Data Bus:** The data bus is used to transfer data between memory and processor or between I/O device and processor. For example, an 8-bit processor will generally have an 8-bit data bus and a 16-bit processor will have 16-bit data bus.
- **Control Bus:** The control bus carry control signals, which consists of signals for selection of memory or I/O device from the given address.



## 2. Functions of Control Unit of 8085.

**Ans:** • **Data Bus:** Data bus carries data in binary form between microprocessor and other external units such as memory. Data bus is **bidirectional** in nature. The data bus width of 8085 microprocessor is 8-bit.  $2^8$  combination of binary digits and are typically identified as D0 – D7.

• **Address Bus:** The address bus carries addresses and is one-way bus from microprocessor to the memory or other devices. 8085 microprocessors contain 16-bit address bus and are generally identified as A0 - A15. the eight data bits (D0 – D7) and hence, they are bidirectional.

• **Control Bus:** The control bus carries control signals partly unidirectional and partly bidirectional. The following control and status signals are used by 8085 processor:

I. ALE (Address Latch Enable): 
$$ALE = \begin{cases} 1 \text{ (Eable)} & AD_0-AD_7 = A_0 - A_7 \text{ (Will carry only address).} \\ 0 \text{ (disable)} & AD_0-AD_7 = D_0 - D_7 \text{ (Will carry only data).} \end{cases}$$

II. RD (active low output): when the RD = 0, it will do read operation.

III. WR (active low output): when the WR = 0, it will do write operation.

IV. IO/M : It is a signal that distinguishes between a memory operation and an I/O operation.

V. S1 and S0: These are status signals used to specify the type of operation being performed; they are listed in Table 1.

Table 1 Status signals and associated operations.

| S1 | S0 | States |
|----|----|--------|
| 0  | 0  | Halt   |
| 0  | 1  | Write  |
| 1  | 0  | Read   |
| 1  | 1  | Fetch  |

## Pdf – 4

### 1. Function of control Unit of 8086.

• **Data bus:** Carries data in binary form between microprocessor and other external units and vise-versa. The data bus width of 8086 microprocessor is 16-bit.  $2^{16}$  combination of binary digits and are typically identified as D0 – D15.

• **Address Bus:** It contain 20 bit address bus and identified as A0-A19.

○ **(AD0 –AD15) [Address/data bus]:** address bus are multiplexed with (D0-D15) data bus.

○ **(A16/S3 –A19/S6) [Address/status bus]:** there are 4 address/status bus.

• **Control bus:** Carries control signals. The following control and status signals used in 8086 microprocessor.

#### Pin in max mode-

- BHE/S7- bus high enable is an active low signal, so it active when BHE= 0.
- RD- this is used for read operation when RD = 0.

○  $\overline{MN}/\overline{MX}$  -  $\begin{cases} 1, \text{ maximum mode} \\ 0, \text{ minimum mode} \end{cases}$

#### Pin in min mode-

- **WR**- 0, then write operation active.
- **ALE**- address appears on (AD0-AD15).

○  $\overline{IO}/\overline{M}$  -  $\begin{cases} 0, \text{ I/O operation} \\ 1, \text{ memory operation} \end{cases}$

## Pdf- 5

### 1. What is Addressing Modes?

**Ans:** The various formats of specifying the operands are called addressing modes.

### 2. Memory Addressing Modes.

In this addressing mode, one operand (source or destination) of an instruction must be an address of a memory location, which will be calculated by PA.

**PA= [Segment] x 10H + Offset**

**Example:** Offset or EA= [ARRAY] + [BX] + [SI] = 2AH + 5000H + 2000H = 702AH



### 3. Explain Six types of memory addressing mode.

**i. Direct or Displacement Addressing mode:** In this mode, the EA is directly given in the instruction.

**Example:** MOV AX, [0500]; EA=0500H, AX will get data from 0500H address.

**ii. Register Indirect Addressing mode:** In this mode, the EA is in SI, DI or BX, which is calculated by PA.

**PA= [DS] x 10H+EA**

**Example:** MOV AL, [SI]; Calculated PA=21200H is the address of memory location.

**iii. Based Addressing mode:** In this mode, the EA is calculated by adding the contents of BX or BP and a displacement (variable).

**So, PA= [DS]x10H+EA**

**Example:** MOV BL,ARRAY[BX]; Calculated PA=32105H is the address of memory location

**iv. Indexed Addressing mode:** In this mode, the EA is calculated by adding the contents of SI or DI and a displacement (variable).

**So, PA= [DS]x10H+EA**

**Example:** MOV AL,START[SI]; Calculated PA=12403H is the address of memory location

**v. Based Indexed Addressing mode:** In this mode, the EA is calculated by adding the contents of BX or BP, SI or DI and a displacement (variable).

**So, PA= [DS]x10H+EA**

**Example:** MOV CL, DISPLACEMENT[BX][SI]; Calculated PA=81A07H is the address of memory location

**vi. String Addressing mode:** This addressing mode is related to string instructions.

**Example:** MOVS B ; MOVS Move the string, B Byte  
MOVS W ; MOVS Move the string, W Word

### 4. Math.

**\*\*Suppose, [SI]=0400H, [DI]=0500H, [DF]=1, [DS]=3000H, [ES]=5000H, [30400H]=02H, [50500H]=05H. Justify the following instruction:  
MOVS B.**

**Solution:** In this addressing mode, the following steps will be followed:

- At first, the two physical addresses (PA1 and PA2) will be calculated:  
 $PA1 = [DS] \times 10H + [SI]$  and  $PA2 = [ES] \times 10H + [DI]$
- Assign the PAs values from the given values.
- The contents of PA1 will be moved into PA2.
- Write the revised values of PA1 and PA2 after the instruction will be executed.
- Finally, the revised values of SI and DI will be written according to the given value of DF.

**Solve the following Home Works (HW):**

Now,  $PA1 = [DS] \times 10H + [SI] = 30400H$   
 $PA2 = [ES] \times 10H + [DI] = 50500H$

**Before instruction execution,**

$PA1 = [30400H] = 02H$  and  $PA2 = [50500H] = 05H$

**After instruction execution,**

$PA1 = [30400H] = 02H$  and  $PA2 = [50500H] = 02H$

**Before instruction execution,**

$[SI] = 0400H$  and  $[DI] = 0500H$

**After instruction execution,**

$[SI] = 0399H$  and  $[DI] = 0499H$ ; Because  $DF=1$

**1. Calculate the Physical Addresses with comments of the following instructions:**

i). MOV [BX],AL; ii). MOV BL,[SI]; where  $[DS]=3000H$ ,  $[BX]=250AH$  and  $[SI]=0300H$ .

**2. Mention the name of addressing modes, find out PAs of the following instructions:**

i). MOV AL,START[BX]; ii). MOV BETA[DI],CL and MOV AL,ARRAY[BX][SI]  
Where  $[DS]=2000H$ ,  $[BX]=2A25H$ ,  $[SI]=0700H$ ,  $[DI]=0200H$ ,  $[START]=02H$ ,  $[BETA]=03H$  and  $[ARRAY]=04H$ .

**3. Justify the below instruction with the name of addressing mode:**

i). MOVS B

Assume the following data prior to execution of the above instruction:

$[SI]=0300H$ ,  $[DI]=0400H$ ,  $[DF]=0$ ,  $[BX]=5000H$ ,  $[DS]=2000H$ ,  $[ES]=3000H$ ,  $[20300H]=05H$ ,  $[30400H]=09H$ .

## Lab Pdf

### 1. What is Assembly Language?

- **Assembly language** is a low-level language that helps to communicate directly with computer hardware. An assembler is used to convert assembly code into machine language.
- Assembly languages contain **mnemonic codes** that specify what the processor should do. Ex – ADD for addition, SUB for subtraction.

### 2. Syntax of Assembly Language.

Assembly language code is generally **not case sensitive**. but we use upper case to differentiate code from the rest of the text.

#### Statements:

Programs consist of statements, one per line. Each statement is either an **instruction**, which the assembler translates into machine code, or an **assembler directive**.

| name | operation | operand(s) | comment |
|------|-----------|------------|---------|
|------|-----------|------------|---------|

❖ An example of an instruction is :

|        |          |  |                     |
|--------|----------|--|---------------------|
| START: | MOV CX,5 |  | ;initialize counter |
|--------|----------|--|---------------------|

Here, the **name field** consists of the **label START**: The **operation** is **MOV**, the **operands** are **CX and 5**, and the comment is ; initialize counter.

### 3. Name Field.

**Ans:** i. The name field is used for instruction labels, procedure names, and variable names.

ii. Names can be from 1 to 31 characters long, and may consist of letters, digits, and special characters like ?, @, \_, \$, % .

iii. Some Example of legal names : diu\_25, diu@cse, .Test, Done?

iv. Some Example of illegal names : TWO WORDS, 2abc, A45.28, You&ME

### 4. Operation Field.

Ans: MOV, ADD, SUB